`Problem Statement – Practical 1

Student Profile Viewer App – A Basic Android Application Using Activities, Services, and User Interface Components

You are required to design and develop an Android application named Student Profile Viewer. The objective of this practical is to help students understand the basic components of Android such as Activities, Services, Content Providers, basic User Interface elements, and working with an Android Virtual Device (AVD) and USB Debugging.

Procedure:

Open Android Studio.

Click File → New → New Project.

Select Empty Views Activity.

Name: StudentProfileViewer

Package Name: package com.example.studentprofileviewer

Language: Java

Minimum SDK: API 21 (Android 5.0)

Click Finish.

Source Code

Design the Main Layout (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">
        <!-- Student Name -->
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Student Name"
            android:textStyle="bold"
            android:paddingTop="8dp"/>

        <EditText
            android:id="@+id/etName"
            android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
    android:hint="Enter Name"
    android:inputType="textPersonName"/>
<!-- Roll Number -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Roll Number"
    android:textStyle="bold"
    android:paddingTop="8dp"/>
<EditText
    android:id="@+id/etRoll"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Roll Number"
    android:inputType="number"/>
<!-- Course -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Course"
    android:textStyle="bold"
    android:paddingTop="8dp"/>
<EditText
    android:id="@+id/etCourse"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Course"
    android:inputType="text"/>
<!-- Email -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Email"
    android:textStyle="bold"
    android:paddingTop="8dp"/>
<EditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Email"
    android:inputType="textEmailAddress"/>
<!-- Button -->
<Button
    android:id="@+id/btnViewProfile"
    android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:text="View Profile"
        android:paddingTop="12dp"
        android:layout_marginTop="16dp"/>
    </LinearLayout>
</ScrollView>
```

Create Profile Activity

Procedure

1. Right-click app → java → com.example.studentprofileviewer.
2. Select **New → Activity → Empty Activity**.
3. Name: ProfileActivity.
4. Click **Finish**.

Android Studio generates ProfileActivity.java and activity_profile.xml

Layout (activity_profile.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <TextView
        android:id="@+id/tvName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:textStyle="bold"
        android:padding="8dp"/>
    <TextView
        android:id="@+id/tvRoll"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:padding="8dp"/>
    <TextView
        android:id="@+id/tvCourse"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:padding="8dp"/>
    <TextView
        android:id="@+id/tvEmail"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:padding="8dp"/>
</LinearLayout>
```

Code (<u>ProfileActivity.java</u>)

```
package com.example.studentprofileviewer;
import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class ProfileActivity extends AppCompatActivity {
    TextView tvName, tvRoll, tvCourse, tvEmail;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_profile);
        // Initialize TextViews
        tvName = findViewById(R.id.tvName);
        tvRoll = findViewById(R.id.tvRoll);
        tvCourse = findViewById(R.id.tvCourse);
        tvEmail = findViewById(R.id.tvEmail);
        // Get data from Intent (sent from MainActivity)
        String name = getIntent().getStringExtra("NAME");
        String roll = getIntent().getStringExtra("ROLL");
        String course = getIntent().getStringExtra("COURSE");
        String email = getIntent().getStringExtra("EMAIL");
        // Set data to TextViews
        tvName.setText("Name: " + name);
        tvRoll.setText("Roll Number: " + roll);
        tvCourse.setText("Course: " + course);
        tvEmail.setText("Email: " + email);
        // Start LoggingService in background
        Intent serviceIntent = new Intent(this, LoggingService.class);
        serviceIntent.putExtra("NAME", name);
        serviceIntent.putExtra("ROLL", roll);
        startService(serviceIntent);
    }
}
```

Add Logging Service

Procedure

1. Right-click app → java → <u>com.example.studentprofileviewer</u>.
2. Select **New → Java Class**

3. Name: LoggingService.
4. Click **Finish**.

```
package com.example.studentprofileviewer;

import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.util.Log;

public class LoggingService extends Service {

    private static final String TAG = "LoggingService";

    @Override
    public void onCreate() {
        super.onCreate();
        Log.d(TAG, "Service Created");
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // Get data passed from ProfileActivity
        String name = intent.getStringExtra("NAME");
        String roll = intent.getStringExtra("ROLL");

        Log.d(TAG, "Profile Viewed: " + name + " (Roll: " + roll + ")");

        // Service runs once and stops (not sticky)
        return START_NOT_STICKY;
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        Log.d(TAG, "Service Destroyed");
    }

    @Override
    public IBinder onBind(Intent intent) {
        // Not using bound service here
        return null;
    }
}
```

MainActivity.java

```
package com.example.studentprofileviewer;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etName, etRoll, etCourse, etEmail;
    Button btnViewProfile;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etName = findViewById(R.id.etName);
        etRoll = findViewById(R.id.etRoll);
        etCourse = findViewById(R.id.etCourse);
        etEmail = findViewById(R.id.etEmail);
        btnViewProfile = findViewById(R.id.btnViewProfile);

        btnViewProfile.setOnClickListener(v -> {
            Intent intent = new Intent(MainActivity.this, ProfileActivity.class);
            intent.putExtra("NAME", etName.getText().toString());
            intent.putExtra("ROLL", etRoll.getText().toString());
            intent.putExtra("COURSE", etCourse.getText().toString());
            intent.putExtra("EMAIL", etEmail.getText().toString());
            startActivity(intent);
        });
    }
}
```

Register Service in Manifest

Open AndroidManifest.xml → inside <application> add:

```
<service android:name=".LoggingService" />
```

Run and Test

Practical:02

2.1: *Create an Android app that defines custom colors in colors.xml and applies them to the background of a TextView*

STEP 1 — Create New Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Views Activity**
4. Click **Next**
5. Fill details:

Project Name: Practical2_1_CustomColors

Package Name: *(auto)*

Language: Java

Minimum SDK: API 21

6. Click **Finish**

Android separates reusable items into folders under **res/**
Examples:

- colors.xml → stores color definitions
- strings.xml → stores text
- styles.xml → themes
- layout.xml → UI structure

This practical focuses on **colors.xml**

STEP 3 — Define Custom Colors in colors.xml

Go to:

app → res → values → colors.xml

Replace everything inside with:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="myBackground">#FF9800</color>   <!-- Orange -->
    <color name="myTextColor">#FFFFFF</color>    <!-- White -->
```

```
</resources>
```

STEP 4 — Design the UI in activity_main.xml

Go to
app → res → layout → activity_main.xml

Replace with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">
    <TextView
        android:id="@+id/txtSample"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello Custom Colors!"
        android:textSize="24sp"
        android:padding="20dp" />
</LinearLayout>
```

Why not set colors here?

Because **practical requires setting color using Java code**.

STEP 5 — Write Java code in MainActivity.java

Go to:
app → java → yourpackagename → MainActivity.java

Replace code with:

```
package com.example.practical2_1_customcolors;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
```

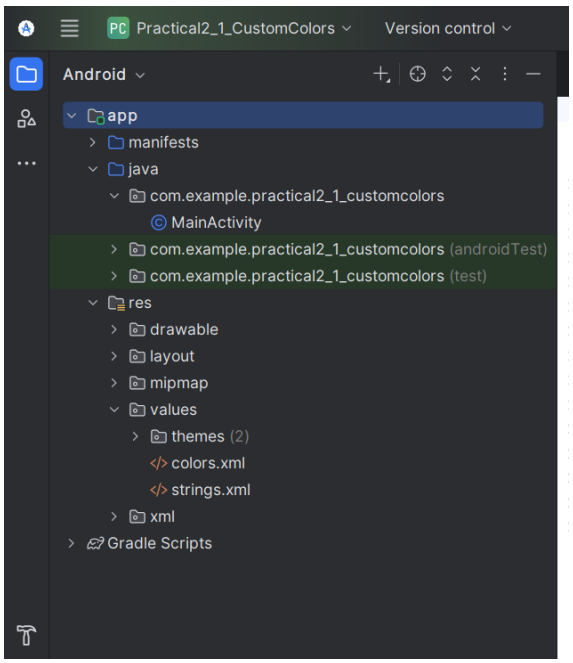
```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Step 1: Find TextView by ID
        TextView txt = findViewById(R.id.txtSample);
        // Step 2: Apply custom colors from colors.xml
        txt.setBackgroundColor(getResources().getColor(R.color.myBackground));
        txt.setTextColor(getResources().getColor(R.color.myTextColor));
    }
}
```

STEP 6 — Run the App

1. Choose **Emulator** or **Physical Device**
2. Press **Run ▶**
3. Output Expected:

A screen displaying:

Orange background TextView
White text inside
Text: **Hello Custom Colors!**



Problem Statement:02

To implement an Android application using resource files to dynamically modify displayed text and background color through user interaction.

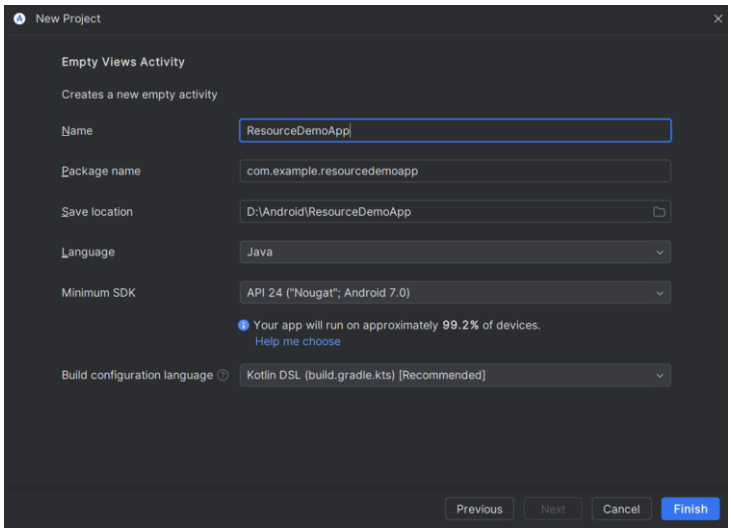
Project Configuration (IMPORTANT)

Fill exactly like this (you may change app name slightly):

- **Name:** ResourceDemoApp
- **Package name:** auto-generated (leave it)
- **Save location:** default
- **Language: Java**
- **Minimum SDK:** API 21 or above

Click **Finish**

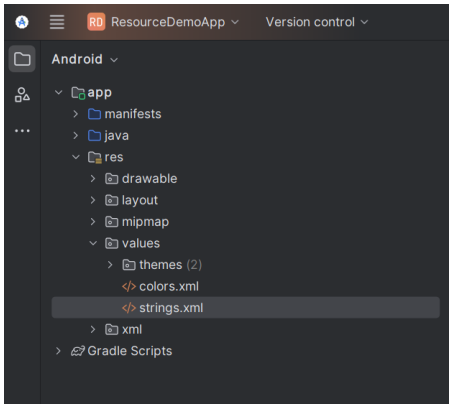
Wait till **Gradle build completes**.



Step3: **Open:**

res → values → strings.xml

(SETUP STRING RESOURCES)



Code:

```
<resources>
    <string name="app_name">ResourceDemoApp</string>
    <!-- Text shown on screen -->
    <string name="light_mode_text">Welcome to Resource Demo</string>
    <string name="dark_mode_text">Dark Mode Activated</string>
    <!-- Button labels -->
    <string name="btn_dark">Switch to Dark Mode</string>
    <string name="btn_light">Switch to Light Mode</string>
</resources>
```

Step:04

res → values → colors.xml

Code:

```
<resources>

    <color name="light_background">#FFFFFF</color>

    <color name="dark_background">#222222</color>

    <color name="light_text">#000000</color>

    <color name="dark_text">#FFFFFF</color>

</resources>
```

Step:05

res → layout → activity_main.xml

Replace everything with this:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:id="@+id/main_layout"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/light_background">
    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/light_mode_text"
        android:textSize="22sp"
        android:textColor="@color/light_text"
        android:layout_centerInParent="true"/>
    <Button
        android:id="@+id/btnToggle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/btn_dark"
        android:layout_centerHorizontal="true"
        android:layout_alignParentBottom="true"
        android:layout_marginBottom="50dp"/>
</RelativeLayout>
```

Step6:Locate: java → your.package.name → MainActivity.java

Code:

```
package com.example.resourcedemoapp;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.Button;
import android.widget.RelativeLayout;
import android.widget.TextView;
public class MainActivity extends AppCompatActivity {
    boolean isLightMode = true;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        RelativeLayout mainLayout = findViewById(R.id.main_layout);
```

```
        TextView tvMessage = findViewById(R.id.tvMessage);
        Button btnToggle = findViewById(R.id.btnToggle);
        btnToggle.setOnClickListener(v -> {
            if (isLightMode) {
                // Switch to Dark Mode

mainLayout.setBackgroundColor(getResources().getColor(R.color.dark_background));
                tvMessage.setText(getString(R.string.dark_mode_text));
                tvMessage.setTextColor(getResources().getColor(R.color.dark_text));
                btnToggle.setText(getString(R.string.btn_light));
                isLightMode = false;
            } else {
                // Switch to Light Mode

mainLayout.setBackgroundColor(getResources().getColor(R.color.light_background));
                tvMessage.setText(getString(R.string.light_mode_text));
                tvMessage.setTextColor(getResources().getColor(R.color.light_text));
                btnToggle.setText(getString(R.string.btn_dark));
                isLightMode = true;
            }

        });
    }
}
```

Now **Run the App**

Practical:2.3

Practical Title

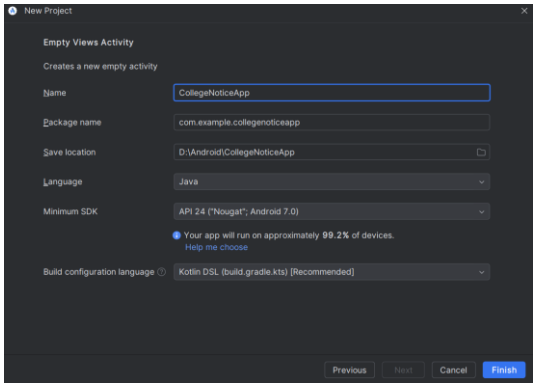
Exploration of Drawable and Dimension Resources in Android

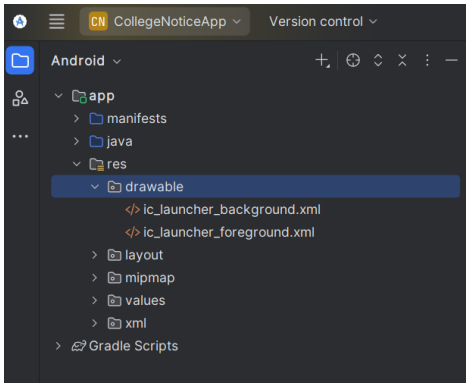
Aim

To study and implement **Drawable** and **Dimension** resources in an Android application for designing a visually consistent and well-spaced user interface.

Problem Statement

To design and develop a simple Android application named **"College Notice Board App"** that demonstrates the use of **drawable** and **dimension** resources.





PRACTICAL: Drawable & Dimension Resources (TCSC)

STEP 1: Open Android Studio

- Click **New Project**

STEP 2: Select Project Template

- Choose **Empty Views Activity**
- Click **Next**

STEP 3: Configure Your Project

Fill exactly like this:

Field	**Value**
Name	CollegeNoticeApp
Package name	com.example.collegenoticeapp
Save location	Default
Language	**Java**
Minimum SDK	API 21 (Android 5.0)
Build configuration language	Default

Click **Finish**
Wait until Gradle build completes.

STEP 4: Add Image (Drawable Resource)

1. Copy your image **logo.jpg**
2. Go to:

app → src → main → res → drawable

3. Paste **logo.jpg** inside drawable

No renaming required

STEP 5: Create Dimension Resources

Open:

res → values → dimens.xml

If not present:

- Right-click values
- New → Values Resource File
- Name: dimens

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="logo_size">120dp</dimen>
    <dimen name="padding_large">16dp</dimen>
    <dimen name="padding_medium">12dp</dimen>
    <dimen name="text_large">20sp</dimen>
    <dimen name="text_medium">16sp</dimen>
</resources>
```

STEP 6: Edit Layout File

Open:

res → layout → activity_main.xml

Replace entire code with this:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="@dimen/padding_large">

    <ImageView
        android:layout_width="@dimen/logo_size"
        android:layout_height="@dimen/logo_size"
        android:src="@drawable/logo"
        android:contentDescription="College Logo"
        android:layout_marginBottom="@dimen/padding_medium" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="TCSC"
        android:textSize="@dimen/text_large"
        android:layout_marginBottom="@dimen/padding_medium" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="College Notice Board"
```

```
        android:textSize="@dimen/text_medium" />
</LinearLayout>
```

STEP 7: Java File

Open:

java → com.example.collegenoticeapp → MainActivity.java

Default code is sufficient:

```
package com.example.collegenoticeapp;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

STEP 8: Run the Application

1. Click **Run**

Practical: 5.1

Problem Statement

Design and develop an Android application that displays an AppBar (Toolbar) at the top of the screen. The AppBar should show the application title and remain visible across the application interface to provide a consistent user experience. The title of the AppBar should be updated dynamically at runtime.

Project Configuration

Project Name: DynamicAppBarDemo

Language: Java

Activity Type: Empty Views Activity

Minimum SDK: API 21

Design Layout with AppBar

Open **res/layout/activity_main.xml**
Replace the entire code with the following:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.coordinatorlayout.widget.CoordinatorLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"

    android:layout_width="match_parent"

    android:layout_height="match_parent">

    <!-- AppBar -->

    <com.google.android.material.appbar.MaterialToolbar

        android:id="@+id/toolbar"

        android:layout_width="match_parent"

        android:layout_height="wrap_content"

        android:background="?attr/colorPrimary"

        android:title="Dynamic AppBar"

        android:titleTextColor="@android:color/white"

        app:layout_scrollFlags="scroll|enterAlways"/>

    <!-- Main Content -->

    <LinearLayout
```

```
        android:orientation="vertical"
        android:padding="20dp"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="56dp">
        <EditText
            android:id="@+id/editTitle"
            android:hint="Enter new AppBar title"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <Button
            android:id="@+id/btnChangeTitle"
            android:text="Change AppBar Title"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
    </LinearLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Explanation

MaterialToolbar → AppBar at the top

EditText → Accepts new title

Button → Updates AppBar title dynamically

AppBar remains **visible at all times**

STEP 3: Write Java Code

Open **MainActivity.java**

```
package com.example.dynamicappbardemo;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
public class MainActivity extends AppCompatActivity {
```

```
    Toolbar toolbar;
    EditText editTitle;
    Button btnChangeTitle;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Initialize AppBar
        toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
        // Initialize UI components
        editTitle = findViewById(R.id.editTitle);
        btnChangeTitle = findViewById(R.id.btnChangeTitle);
        // Dynamic AppBar title change
        btnChangeTitle.setOnClickListener(v -> {
            String newTitle = editTitle.getText().toString();
            if (!newTitle.isEmpty()) {
                getSupportActionBar().setTitle(newTitle);
            }
        });
    }
}
```

STEP 4: Run the Application

Prac:5.2

Aim To design and develop an Android Calculator Application that demonstrates the effective use of AppBar, Fragments, and basic UI components. The application aims to perform basic arithmetic operations based on user input handled within a fragment, while maintaining a consistent AppBar across the application interface to ensure a structured and user-friendly design.

Problem Statement

Design and develop an Android Calculator Application that demonstrates the use of AppBar, Fragments, and basic UI components. The application should perform arithmetic operations using user input handled inside a fragment, while maintaining a consistent AppBar across the application.

Project Details

Project Name: FragmentCalculatorApp

Language: Java

Activity Type: Empty Views Activity

Minimum SDK: API 21

STEP 1: Activity Layout with AppBar + Fragment Container

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <!-- AppBar -->
    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:title="Fragment Calculator"
        android:background="?attr/colorPrimary"
        android:titleTextColor="@android:color/white" />
    <!-- Fragment Container -->
    <FrameLayout
        android:id="@+id/fragment_container"
```

```
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="56dp"/>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Create Fragment

Right click **java → package**

New → Fragment → Fragment (Blank)

Name: fragment_calculator

Click **Finish**

STEP 2: Calculator Fragment Layout (UI Components)

fragment_calculator.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="20dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <EditText
        android:id="@+id/num1"
        android:hint="Enter first number"
        android:inputType="numberDecimal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
    <EditText
        android:id="@+id/num2"
        android:hint="Enter second number"
        android:inputType="numberDecimal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
    <LinearLayout
        android:orientation="horizontal"
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <Button
        android:id="@+id/btnAdd"
        android:text="+"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content"/>
    <Button
        android:id="@+id/btnSub"
        android:text="-"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content"/>
    <Button
        android:id="@+id/btnMul"
        android:text="×"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content"/>
    <Button
        android:id="@+id/btnDiv"
        android:text="÷"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content"/>
</LinearLayout>
<TextView
    android:id="@+id/txtResult"
    android:text="Result will appear here"
```

```
        android:textSize="18sp"

        android:paddingTop="20dp"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"/>

</LinearLayout>
```

STEP 3: Calculator Fragment Logic

CalculatorFragment.java

```
package com.example.fragmentcalculatorapp;
import android.os.Bundle;
import androidx.fragment.app.Fragment;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
public class CalculatorFragment extends Fragment {
    EditText num1, num2;
    TextView txtResult;
    public CalculatorFragment() {}
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
savedInstanceState)
    {

        View view = inflater.inflate(R.layout.fragment_calculator, container, false);
        num1 = view.findViewById(R.id.num1);
        num2 = view.findViewById(R.id.num2);
        txtResult = view.findViewById(R.id.txtResult);
        Button btnAdd = view.findViewById(R.id.btnAdd);
        Button btnSub = view.findViewById(R.id.btnSub);
        Button btnMul = view.findViewById(R.id.btnMul);
        Button btnDiv = view.findViewById(R.id.btnDiv);
        View.OnClickListener listener = v -> calculate(v.getId());
        btnAdd.setOnClickListener(listener);
        btnSub.setOnClickListener(listener);
        btnMul.setOnClickListener(listener);
        btnDiv.setOnClickListener(listener);

        return view;
    }
    private void calculate(int id) {
        double a, b;
```

```
        if (num1.getText().toString().isEmpty() ||
                num2.getText().toString().isEmpty()) {
            txtResult.setText("Please enter both numbers");
            return;
        }
        a = Double.parseDouble(num1.getText().toString());
        b = Double.parseDouble(num2.getText().toString());
        double result = 0;
        if (id == R.id.btnAdd) result = a + b;
        else if (id == R.id.btnSub) result = a - b;
        else if (id == R.id.btnMul) result = a * b;
        else if (id == R.id.btnDiv) {
            if (b == 0) {
                txtResult.setText("Division by zero not allowed");
                return;
            }
            result = a / b;
        }
        txtResult.setText("Result: " + result);
    }
}
```

STEP 4: Load Fragment in Activity

MainActivity.java

```
package com.example.fragmentcalculatorapp;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
        if (savedInstanceState == null) {
            getSupportFragmentManager()
                    .beginTransaction()
                    .add(R.id.fragment_container, new CalculatorFragment())
                    .commit();
        }
    }
}
```

Practical – 6.1

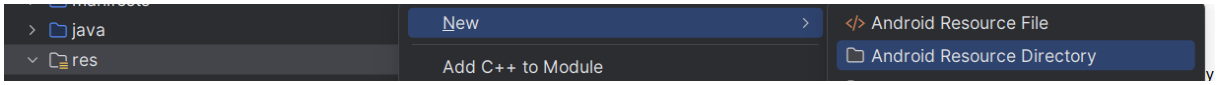
AIM

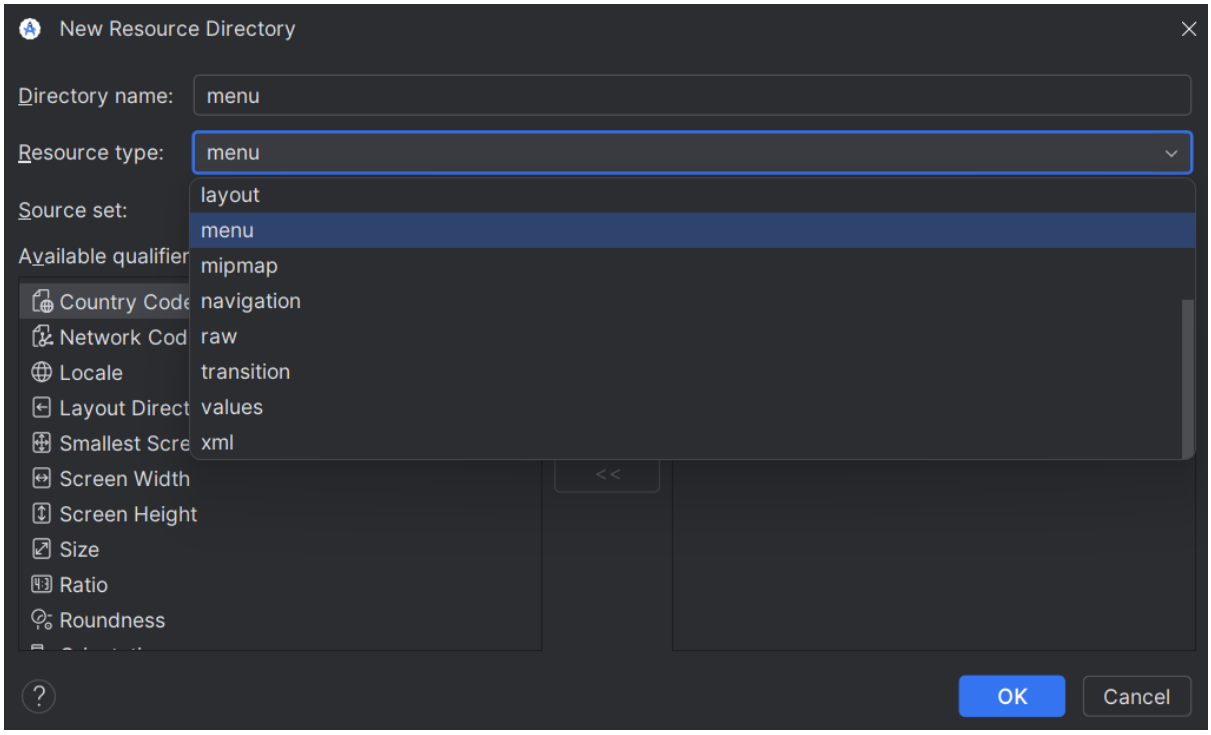
To implement an Android application demonstrating **programming of menus and dialogs** for user interaction.

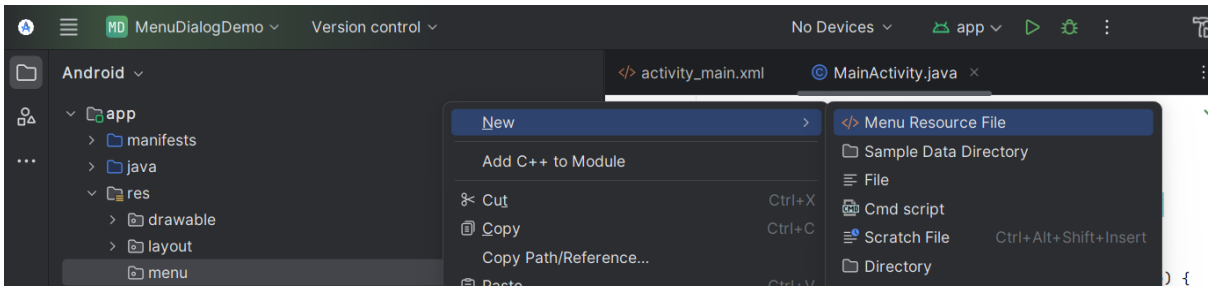
PROBLEM STATEMENT

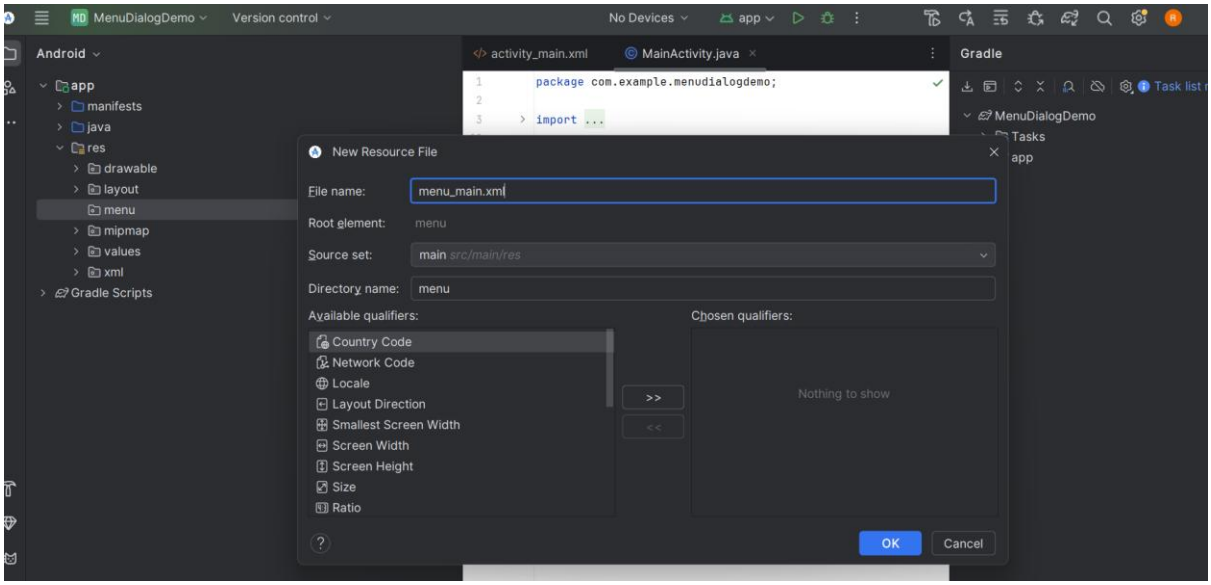
Design and develop an Android application that uses an **Options Menu** with multiple menu items. On selecting a menu item, display an appropriate **dialog** to show information or to confirm user action, and perform the required operation based on the user's response.

```
app
 ├── java
 |   └── com.example.menudialogapp
 |       └── MainActivity.java
 ├── res
 |   ├── layout
 |   |   └── activity_main.xml
 |   ├── menu
 |   |   └── menu_main.xml
 |   └── values
 |       └── strings.xml
```




New Resource Directory
Directory name:
menu
Resource type:
menu
Source set:
Available qualifier
layout
menu
mipmap
navigation
raw
transition
values
xml
Country Code
Network Cod
Locale
Layout Direct
Smallest Scre
Screen Width
Screen Height
Size
Ratio
Roundness
OK
Cancel


MenuDialogDemo
Version control
No Devices
app
Android
activity_main.xml
MainActivity.java
app
manifests
java
res
drawable
layout
menu
New
Add C++ to Module
Cut
Ctrl+X
Copy
Ctrl+C
Copy Path/Reference...
Menu Resource File
Sample Data Directory
File
Cmd script
Scratch File
Ctrl+Alt+Shift+Insert
Directory


MenuDialogDemo
Version control
No Devices
app
Android
activity_main.xml
MainActivity.java
Gradle
package com.example.menudialogdemo;
import ...
app
manifests
java
res
drawable
layout
menu
mipmap
values
xml
Gradle Scripts
MenuDialogDemo
Tasks
app
New Resource File
File name:
menu_main.xml
Root element:
menu
Source set:
main src/main/res
Directory name:
menu
Available qualifiers:
Country Code
Network Code
Locale
Layout Direction
Smallest Screen Width
Screen Width
Screen Height
Size
Ratio
Chosen qualifiers:
Nothing to show
OK
Cancel

activity_main.xml:SourceCode:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <!-- Toolbar -->
    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="?attr/colorPrimary"
        android:title="Menu Dialog App"
        android:titleTextColor="@android:color/white"
        app:popupTheme="@style/ThemeOverlay.AppCompat.Light"/>
    <!-- Content -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center"
        android:orientation="vertical">
        <TextView
            android:id="@+id/txtMessage"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Select an option from menu"
            android:textSize="20sp"
            android:textStyle="bold"/>
    </LinearLayout>
```

```
</LinearLayout>
```

MainActivity.java

```
package com.example.menudialogapp;
import android.content.DialogInterface;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.TextView;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
public class MainActivity extends AppCompatActivity {
    TextView txtMessage;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Connect Toolbar to Action Bar
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
        // Connect TextView
        txtMessage = findViewById(R.id.txtMessage);
    }
    // Inflate Options Menu
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    // Handle Menu Item Clicks
    @Override
```

```
    public boolean onOptionsItemSelected(MenuItem item) {
        int itemId = item.getItemId();
        if (itemId == R.id.menu_about) {
            showAboutDialog();
            return true;
        }
        if (itemId == R.id.menu_reset) {
            showResetDialog();
            return true;
        }
        if (itemId == R.id.menu_exit) {
            showExitDialog();
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    // About Dialog
    private void showAboutDialog() {
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle("About App");
        builder.setMessage("This application demonstrates the use of Options Menu and Alert Dialogs in
Android.");
        builder.setPositiveButton("OK", null);
        builder.show();
    }
    // Reset Confirmation Dialog
    private void showResetDialog() {
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle("Confirm Reset");
        builder.setMessage("Do you want to reset the displayed text?");
        builder.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
```

```
            @Override
            public void onClick(DialogInterface dialog, int which) {
                txtMessage.setText("Text has been reset");
            }
        });
        builder.setNegativeButton("No", null);
        builder.show();
    }
    // Exit Confirmation Dialog
    private void showExitDialog() {
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle("Exit Application");
        builder.setMessage("Are you sure you want to exit?");
        builder.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                finish();
            }
        });
        builder.setNegativeButton("No", null);
        builder.show();
    }
}
```

Menu_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/menu_about"
        android:title="About App"/>
    <item
        android:id="@+id/menu_reset"
        android:title="Reset Text"/>
    <item
        android:id="@+id/menu_exit"
```

```
    android:title="Exit App"/>
</menu>
```

Prac:07

Problem Statement:

Develop an Android application for a College Event Registration system using Intents, Events, Listeners, and Adapters. The first activity should display a list of events using a ListView or RecyclerView with an Adapter. On selecting an event, the application must handle the click using a Listener and navigate to the second activity through an Intent while passing the event name. The second activity should display the selected event and contain a Register button which, on click, shows a confirmation message using Toast or AlertDialog.

Aim:

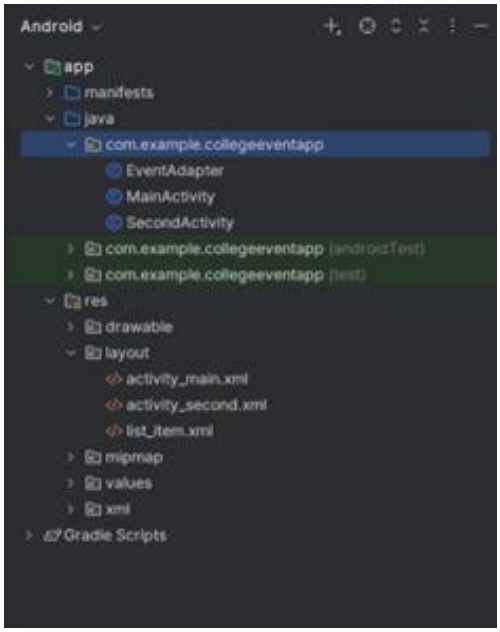
To develop an Android application that demonstrates the implementation of Intents, Events, Listeners, and Adapters by creating a College Event Registration system that enables navigation between activities, handles user interactions, and displays data using Adapter-based views.

1. Create New Project in Android Studio

Steps

1. Open Android Studio → *New Project*
2. Select → **Empty Activity**
3. Name: **CollegeEventApp**
4. Language: **Java**
5. Minimum SDK: **21 or above**
6. Finish

2. Project Structure



3. XML Layouts

(A) activity_main.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

(B) list_item.xml

```
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/txtEvent"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:padding="12dp"/>
```

(C) activity_second.xml

```
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="vertical"
   android:gravity="center"
   android:layout_width="match_parent"
   android:layout_height="match_parent">
   <TextView
      android:id="@+id/txtSelected"
      android:textSize="20sp"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"/>
   <Button
      android:id="@+id/btnRegister"
      android:text="Register"
      android:layout_marginTop="20dp"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"/>
</LinearLayout>
```

4. Java Code

(A) EventAdapter.java – ADAPTER

```
package com.example.collegeeventapp;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.BaseAdapter;
public class EventAdapter extends BaseAdapter {
   Context context;
```

```
    String[] events;
    public EventAdapter(Context context, String[] events) {
        this.context = context;
        this.events = events;
    }
    @Override
    public int getCount() {
        return events.length;
    }
    @Override
    public Object getItem(int i) {
        return events[i];
    }
    @Override
    public long getItemId(int i) {
        return i;
    }
    @Override
    public View getView(int i, View view, ViewGroup parent) {
        if(view == null) {
            view = LayoutInflater.from(context)
                    .inflate(R.layout.list_item, parent, false);
        }
        TextView txt = view.findViewById(R.id.txtEvent);
        txt.setText(events[i]);
        return view;
    }
}
```

(B) MainActivity.java – INTENT + LISTENER

```
package com.example.collegeeventapp;
import android.content.Intent;
```

```
import android.os.Bundle;
import android.widget.AdapterView;
import android.widget.ListView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    ListView listView;
    String[] events = {
        "Coding Contest",
        "Technical Quiz",
        "Project Expo",
        "Workshop"
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        listView = findViewById(R.id.listView);
        EventAdapter adapter =
            new EventAdapter(this, events);
        listView.setAdapter(adapter);
        // EVENT + LISTENER
        listView.setOnItemClickListener(
            new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> parent,
                    android.view.View view,
                    int position, long id) {
                // INTENT
                Intent i = new Intent(
                    MainActivity.this,
                    SecondActivity.class);
```

```
            i.putExtra("event",
                    events[position]);
            startActivity(i);
        }
    });
  }
}
```

(C) SecondActivity.java – EVENT HANDLING

```
package com.example.collegeeventapp;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class SecondActivity extends AppCompatActivity {
  TextView txt;
  Button btn;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_second);
    txt = findViewById(R.id.txtSelected);
    btn = findViewById(R.id.btnRegister);
    // RECEIVE INTENT DATA
    String event =
      getIntent().getStringExtra("event");
    txt.setText("Selected: " + event);
    // LISTENER EVENT
    btn.setOnClickListener(v ->
      Toast.makeText(
        getApplicationContext(),
```

```
            "Registered for " + event,
            Toast.LENGTH_LONG
        ).show()
    );
  }
}
```

Paste this inside application in manifest

```
<activity
  android:name="SecondActivity"
  android:exported="false" />
```

Practical:8.1: Services

Aim-To study and implement an Android Service that performs a background task (such as a counter or timer) without direct user interaction.

Problem Statement – Develop an Android application that implements a Started Service which runs a background counter (or timer) and logs the count every second using Logcat.

Field	**Value**
Name	StartedServiceCounter
Package Name	com.example.startedservicecounter
Language	**Java**
Minimum SDK	API 21 (or default)

Click Finish

Wait for Gradle sync.

STEP 2: Create Service Class

Right Click on:

app > java > com.example.startedservicecounter

New → Java Class
Name it: CounterService

STEP 3: Write CounterService.java

Replace everything with this:

```
package com.example.startedservicecounter;

import android.app.Service;

import android.content.Intent;
```

```
import android.os.Handler;
import android.os.IBinder;
import android.util.Log;
import android.widget.Toast;
public class CounterService extends Service {
    private Handler handler;
    private Runnable runnable;
    private int counter = 0;
    private boolean isRunning = false;
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        if (isRunning) {
            Log.d("CounterService", "Service already running");
            return START_STICKY;
        }
        isRunning = true;
        handler = new Handler();
        runnable = new Runnable() {
            @Override
            public void run() {
                counter++;
                Log.d("CounterService", "Counter: " + counter);
                handler.postDelayed(this, 1000);
            }
        };
        handler.post(runnable);
        Toast.makeText(this, "Service Started", Toast.LENGTH_SHORT).show();
        return START_STICKY;
    }
    @Override
    public void onDestroy() {
```

```
        super.onDestroy();


        if (handler != null && runnable != null) {
            handler.removeCallbacks(runnable);
        }
        isRunning = false;
        counter = 0;
        Toast.makeText(this, "Service Stopped", Toast.LENGTH_SHORT).show();
        Log.d("CounterService", "Service Destroyed");
    }
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

STEP 4: Register Service in AndroidManifest.xml

Open:

app > manifests > AndroidManifest.xml

Inside <application> tag add this:

```
<service
    android:name=".CounterService"
    android:enabled="true"
    android:exported="false"/>
```

Very important step
Without this → app crashes.

STEP 5: Design Layout

Open:res > layout > activity_main.xml

Replace everything with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">
    <Button
        android:id="@+id/btnStart"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Service"/>
    <Button
        android:id="@+id/btnStop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop Service"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

STEP 6: Write MainActivity.java

Open:MainActivity.java

Replace everything with:

```
package com.example.startedservicecounter;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    Button btnStart, btnStop;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);


        btnStart = findViewById(R.id.btnStart);
        btnStop = findViewById(R.id.btnStop);
        btnStart.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(MainActivity.this, CounterService.class);
                startService(intent);
            }
        });
        btnStop.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(MainActivity.this, CounterService.class);
                stopService(intent);
            }
        });
    }
}
```

STEP 7: Run the App

Practical:8.2

Aim :To study and implement Android Notification to alert users about application events.

Problem Statement :Develop an Android application that generates a Notification when a button is clicked.

Project Name: Notification

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
    <Button
        android:id="@+id/btnNotify"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Generate Notification" />
</LinearLayout>
```

MainActivity.java

```
package com.example.notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;
public class MainActivity extends AppCompatActivity {
    private static final String CHANNEL_ID = "notification_channel";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btnNotify = findViewById(R.id.btnNotify);
        // Create Notification Channel (Required for Android 8+)
        createNotificationChannel();
        btnNotify.setOnClickListener(view -> {
            // Intent to open app when notification clicked
            Intent intent = new Intent(MainActivity.this, MainActivity.class);
            intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
            PendingIntent pendingIntent = PendingIntent.getActivity(
                    MainActivity.this,
                    0,
                    intent,
                    PendingIntent.FLAG_IMMUTABLE
            );
            // Build Notification
            NotificationCompat.Builder builder =
```

```
            new NotificationCompat.Builder(MainActivity.this, CHANNEL_ID)
                .setSmallIcon(android.R.drawable.ic_dialog_info)
                .setContentTitle("Notification Alert")
                .setContentText("Button Clicked! This is your notification.")
                .setPriority(NotificationCompat.PRIORITY_DEFAULT)
                .setContentIntent(pendingIntent)
                .setAutoCancel(true);
        // Show Notification
        NotificationManager notificationManager =
            (NotificationManager) getSystemService(NOTIFICATION_SERVICE);
        notificationManager.notify(1, builder.build());
    });
  }
  // Method to create Notification Channel
  private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = "Notification Channel";
        String description = "Channel for app notifications";
        int importance = NotificationManager.IMPORTANCE_DEFAULT;
        NotificationChannel channel =
            new NotificationChannel(CHANNEL_ID, name, importance);
        channel.setDescription(description);
        NotificationManager manager =
            getSystemService(NotificationManager.class);
        if (manager != null) {
            manager.createNotificationChannel(channel);
        }
    }
  }
}
```

AndroidManifest.xml

Make sure this line exists:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
  <application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Notification">
    <activity
```

```
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
          <action android:name="android.intent.action.MAIN" />
          <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>
    </application>
</manifest>
```

Final Output When you:

Run app

Practical:09

Aim: To study database programming using SQLite in Android.

Problem Statement: Develop an Android application using SQLite database to create a table and perform insert, update, delete and display operations on the stored records.

Project Details

Field	**Value**
Name	SQLiteDemo
Language	Java
Min SDK	21

Click **Finish**

Wait for Gradle to build.

STEP 2 — How SQLite Works in Android

Android gives a helper class called:

SQLiteOpenHelper

We must create our own class extending this.

This class:Creates database

Creates table

Handles version changes

STEP 3 — Create DatabaseHelper Class

Right click:

app > java > yourpackage

New → Java Class
Name it: DatabaseHelper

DatabaseHelper.java

```
package com.example.sqlitedemo;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
public class DatabaseHelper extends SQLiteOpenHelper {
    public static final String DATABASE_NAME = "Student.db";
    public static final String TABLE_NAME = "student";
    public static final String COL_1 = "ID";
    public static final String COL_2 = "NAME";
    public static final String COL_3 = "MARKS";
    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, 1);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE_NAME +
                " (ID INTEGER PRIMARY KEY AUTOINCREMENT, NAME TEXT, MARKS INTEGER)");
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }
    public boolean insertData(String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, marks);
```

```
        long result = db.insert(TABLE_NAME, null, contentValues);
        return result != -1;
    }
    public Cursor getAllData() {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }
    public boolean updateData(String id, String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_1, id);
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, marks);
        db.update(TABLE_NAME, contentValues, "ID = ?", new String[]{id});
        return true;
    }
    public Integer deleteData(String id) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_NAME, "ID = ?", new String[]{id});
    }
}
```

STEP 4 — Design Layout (activity_main.xml)

Replace with:

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
```

```
android:padding="20dp">
<EditText
    android:id="@+id/editId"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter ID" />
<EditText
    android:id="@+id/editName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name" />
<EditText
    android:id="@+id/editMarks"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Marks" />
<Button
    android:id="@+id/btnAdd"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Insert" />

<Button
    android:id="@+id/btnView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="View All" />
<Button
    android:id="@+id/btnUpdate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
        android:text="Update" />
    <Button
        android:id="@+id/btnDelete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Delete" />
  </LinearLayout>
</ScrollView>
```

STEP 5 — MainActivity.java

Replace code with:

```
package com.example.sqlitedemo;
import androidx.appcompat.app.AppCompatActivity;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
public class MainActivity extends AppCompatActivity {
  DatabaseHelper myDb;
  EditText editName, editMarks, editId;
  Button btnAddData, btnViewAll, btnUpdate, btnDelete;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    myDb = new DatabaseHelper(this);
    editId = findViewById(R.id.editId);
    editName = findViewById(R.id.editName);
    editMarks = findViewById(R.id.editMarks);
    btnAddData = findViewById(R.id.btnAdd);
```

```
    btnViewAll = findViewById(R.id.btnView);
    btnUpdate = findViewById(R.id.btnUpdate);
    btnDelete = findViewById(R.id.btnDelete);
    AddData();
    viewAll();
    UpdateData();
    DeleteData();
}
public void AddData() {
    btnAddData.setOnClickListener(v -> {
        boolean isInserted = myDb.insertData(
                editName.getText().toString(),
                editMarks.getText().toString()
        );
        if (isInserted)
            Toast.makeText(MainActivity.this, "Data Inserted", Toast.LENGTH_LONG).show();
        else
            Toast.makeText(MainActivity.this, "Data Not Inserted", Toast.LENGTH_LONG).show();
    });
}

public void viewAll() {
    btnViewAll.setOnClickListener(v -> {
        Cursor res = myDb.getAllData();
        if (res.getCount() == 0) {
            Toast.makeText(MainActivity.this, "No Data Found", Toast.LENGTH_LONG).show();
            return;
        }
        StringBuffer buffer = new StringBuffer();
        while (res.moveToNext()) {
            buffer.append("ID :" + res.getString(0) + "\n");
```

```
            buffer.append("Name :" + res.getString(1) + "\n");
            buffer.append("Marks :" + res.getString(2) + "\n\n");
        }
        Toast.makeText(MainActivity.this, buffer.toString(), Toast.LENGTH_LONG).show();
    });
  }
  public void UpdateData() {
    btnUpdate.setOnClickListener(v -> {
        boolean isUpdate = myDb.updateData(
                editId.getText().toString(),
                editName.getText().toString(),
                editMarks.getText().toString()
        );
        if (isUpdate)
            Toast.makeText(MainActivity.this, "Data Updated", Toast.LENGTH_LONG).show();
        else
            Toast.makeText(MainActivity.this, "Data Not Updated", Toast.LENGTH_LONG).show();
    });
  }
  public void DeleteData() {
    btnDelete.setOnClickListener(v -> {
        Integer deletedRows = myDb.deleteData(editId.getText().toString());
        if (deletedRows > 0)
            Toast.makeText(MainActivity.this, "Data Deleted", Toast.LENGTH_LONG).show();
        else
            Toast.makeText(MainActivity.this, "Data Not Deleted", Toast.LENGTH_LONG).show();
    });
  }
}
```

Run The App

Test:

Insert → Enter name & marks

View → Shows all records

Update → Enter ID + new values

Delete → Enter ID

Concept	Meaning
SQLiteOpenHelper	Manages database
onCreate()	Creates table
insert()	Adds data
rawQuery()	Reads data
update()	Modifies data
delete()	Removes data

Practical10

Runtime Camera Permission in Android (Java)

Aim: To study and implement security features and runtime permission handling in Android applications.

Problem Statement: Develop an Android application that requests Camera permission at runtime. When the user clicks a button, the application should: Check whether the Camera permission is granted. If not granted, request the permission from the user. Display a message indicating whether the permission is granted or denied. The application should handle permission approval and denial properly.

From Android 6.0 (Marshmallow) onwards, permissions like Camera, Location, etc. must be requested at runtime, not only in the Manifest.

There are 3 main steps in runtime permission handling:

Declare permission in AndroidManifest.xml

Check permission using ContextCompat.checkSelfPermission()

Request permission using ActivityCompat.requestPermissions()

Handle result inside onRequestPermissionsResult()

We will implement exactly this.

Project Setup

Project Name: CameraPermissionApp

Language: Java
Template: Empty Views Activity
Minimum SDK: API 23 or above

1. AndroidManifest.xml

Add Camera permission inside <manifest> but outside <application>.

```
<uses-permission android:name="android.permission.CAMERA"/>
```

2. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/btnCamera"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Request Camera Permission"/>
</LinearLayout>
```

3. MainActivity.java

```
package com.example.camerapermissionapp;
import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
public class MainActivity extends AppCompatActivity {
    private static final int CAMERA_REQUEST_CODE = 100;
```

```
Button btnCamera;
@Override
protected void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_main);
   btnCamera = findViewById(R.id.btnCamera);
   btnCamera.setOnClickListener(v -> checkCameraPermission());
}
private void checkCameraPermission() {
   // Step 1: Check if permission already granted
   if (ContextCompat.checkSelfPermission(this,
         Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED) {

      Toast.makeText(this,
            "Camera Permission Already Granted",
            Toast.LENGTH_SHORT).show();
   } else {
      // Step 2: Request permission
      ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.CAMERA},
            CAMERA_REQUEST_CODE);
   }
}
// Step 3: Handle user response
@Override
public void onRequestPermissionsResult(int requestCode,
                                       @NonNull String[] permissions,
                                       @NonNull int[] grantResults) {
   super.onRequestPermissionsResult(requestCode, permissions, grantResults);
   if (requestCode == CAMERA_REQUEST_CODE) {
      if (grantResults.length > 0
```

```
                && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            Toast.makeText(this,
                    "Camera Permission Granted",
                    Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this,
                    "Camera Permission Denied",
                    Toast.LENGTH_SHORT).show();
        }
    }
  }
}
```

Prac:4

Practical No:4.1

Aim: To design a simple Android application using Linear Layout to display student information and understand vertical and horizontal orientation.

Problem Statement:
Design an Android application that displays student details such as Name, Roll Number, Course, and College Name using Linear Layout. Arrange the views vertically and horizontally to understand orientation.

Step 1: Create New Project

Open **Android Studio**

Click **New Project**

Select **Empty Views Activity**

Click **Next**

Fill details:

Name: StudentProfileLinearLayout

Package: com.example.studentprofile

Language: **Java**

Click **Finish**

Step 2: Open Layout File

app > res > layout > activity_main.xml

Switch to **Code view** (XML only)

Source Code:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <!-- Title -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Student Profile"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_gravity="center"
        android:paddingBottom="20dp" />
    <!-- Name Row -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Name: "
            android:textStyle="bold" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Nikhil" />
    </LinearLayout>
    <!-- Roll Number Row -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingTop="10dp">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Roll No: "
            android:textStyle="bold" />
        <TextView
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="21MCA001" />
    </LinearLayout>
    <!-- Course Row -->
    <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:orientation="horizontal"
      android:paddingTop="10dp">
      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Course: "
        android:textStyle="bold" />
      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="BSc IT" />
    </LinearLayout>
    <!-- College Row -->
    <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:orientation="horizontal"
      android:paddingTop="10dp">
      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="College: "
        android:textStyle="bold" />
      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="ABC Institute of Technology" />
    </LinearLayout>
</LinearLayout>
```

Step 4: Java File (No Code Needed)

Open:

MainActivity.java

No changes required because:
This is **static UI only**, no button or logic.

Step 5: Run the App

Practical 4.2

Aim: To design a simple login screen using Relative Layout and understand positioning of views relative to parent and other views.

Problem Statement

Create an Android application to design a Login Screen using Relative Layout with Username, Password, and Login Button positioned relative to each other.

Project Name: LoginRelativeLayout

activity_main.xml: Source Code

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp">
    <!-- Title -->
    <TextView
        android:id="@+id/txtTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_centerHorizontal="true" />
    <!-- Username -->
    <EditText
        android:id="@+id/edtUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username"
        android:layout_below="@id/txtTitle"
        android:layout_marginTop="20dp" />
    <!-- Password -->
    <EditText
        android:id="@+id/edtPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"
        android:layout_below="@id/edtUsername"
        android:layout_marginTop="10dp" />
```

```
    <!-- Login Button -->
    <Button
        android:id="@+id/btnLogin"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:layout_below="@id/edtPassword"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="20dp" />
</RelativeLayout>
```

MainActivity.java

```
package com.example.loginrelativelayout;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Run

Practical 4.3

Problem Statement

Develop an Android application that displays a student mark sheet using **Table Layout**, showing subject names and corresponding marks in rows and columns.

Project Name: MarkSheetTableLayout

Source Code: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:padding="16dp"
    android:stretchColumns="1">
    <!-- Title -->
    <TableRow>
        <TextView
            android:layout_span="2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Student Mark Sheet"
            android:textSize="22sp"
            android:textStyle="bold"
            android:gravity="center" />
    </TableRow>
    <!-- Header Row -->
    <TableRow>
        <TextView
            android:text="Subject"
            android:textStyle="bold" />
        <TextView
            android:text="Marks"
            android:textStyle="bold" />
    </TableRow>
    <!-- Row 1 -->
    <TableRow>
        <TextView android:text="Mathematics" />
        <TextView android:text="85" />
    </TableRow>
    <!-- Row 2 -->
    <TableRow>
        <TextView android:text="Computer Science" />
        <TextView android:text="90" />
    </TableRow>
    <!-- Row 3 -->
    <TableRow>
        <TextView android:text="Physics" />
        <TextView android:text="78" />
    </TableRow>
    <!-- Row 4 -->
    <TableRow>
        <TextView android:text="English" />
        <TextView android:text="88" />
    </TableRow>
</TableLayout>
```

Run

Prac-4.4

Problem Statement

Develop an Android application that displays a list of contact names entered by the user using **ListView**.

Project Name: ContactListViewApp

activity_main.xml

SourceCode:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="16dp">
   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Contact List"
      android:textSize="22sp"
      android:textStyle="bold"
      android:layout_gravity="center"
      android:paddingBottom="10dp"/>
   <EditText
      android:id="@+id/edtName"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter contact name"/>
   <Button
      android:id="@+id/btnAdd"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Add"
      android:layout_gravity="center"
      android:paddingTop="10dp"/>
   <ListView
      android:id="@+id/listView"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:paddingTop="10dp"/>
</LinearLayout>
```

MainActivity.java

SourceCode:

```
package com.example.contactlistviewapp;
import android.os.Bundle;
```

```
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;
public class MainActivity extends AppCompatActivity {
   EditText edtName;
   Button btnAdd;
   ListView listView;
   ArrayList<String> contactList;
   ArrayAdapter<String> adapter;
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      edtName = findViewById(R.id.edtName);
      btnAdd = findViewById(R.id.btnAdd);
      listView = findViewById(R.id.listView);
      contactList = new ArrayList<>();
      adapter = new ArrayAdapter<>(
            this,
            android.R.layout.simple_list_item_1,
            contactList
      );
      listView.setAdapter(adapter);
      btnAdd.setOnClickListener(new View.OnClickListener() {
         @Override
         public void onClick(View view) {
            String name = edtName.getText().toString();
            if (!name.isEmpty()) {
               contactList.add(name);
               adapter.notifyDataSetChanged();
               edtName.setText("");
            }
         }
      });
   }
}
```

Click **Run**

Prac-4.5

Problem Statement

Design an Android application using **GridView** that allows the user to **enter items dynamically** and display them in a **grid format** on the screen.

Aim

To create an Android application that uses **GridView with Adapter** to display user-entered items dynamically in a grid arrangement.

Project Name: DynamicGridViewApp

activity_main.xml

source Code:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="16dp">
   <EditText
      android:id="@+id/etItem"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter item" />
   <Button
      android:id="@+id/btnAdd"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="Add to Grid" />
   <GridView
      android:id="@+id/gridView"
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:numColumns="2"
      android:horizontalSpacing="10dp"
      android:verticalSpacing="10dp" />
</LinearLayout>
```

MainActivity.java

SourceCode:

```
package com.example.dynamicgridviewapp;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.GridView;
import androidx.appcompat.app.AppCompatActivity;

import java.util.ArrayList;
```

```
public class MainActivity extends AppCompatActivity {
    EditText etItem;
    Button btnAdd;
    GridView gridView;
    ArrayList<String> itemList;
    ArrayAdapter<String> adapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        etItem = findViewById(R.id.etItem);
        btnAdd = findViewById(R.id.btnAdd);
        gridView = findViewById(R.id.gridView);
        itemList = new ArrayList<>();
        adapter = new ArrayAdapter<>(
                this,
                android.R.layout.simple_list_item_1,
                itemList
        );
        gridView.setAdapter(adapter);
        btnAdd.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String item = etItem.getText().toString();
                if (!item.isEmpty()) {
                    itemList.add(item);
                    adapter.notifyDataSetChanged();
                    etItem.setText("");
                }
            }
        });
    }
}
```

Run

Problem Statement:

Develop an Android application for a College Event Registration system using Intents, Events, Listeners, and Adapters. The first activity should display a list of events using a ListView or RecyclerView with an Adapter. On selecting an event, the application must handle the click using a Listener and navigate to the second activity through an Intent while passing the event name. The second activity should display the selected event and contain a Register button which, on click, shows a confirmation message using Toast or AlertDialog.

Aim:

To develop an Android application that demonstrates the implementation of Intents, Events, Listeners, and Adapters by creating a College Event Registration system that enables navigation between activities, handles user interactions, and displays data using Adapter-based views.

1. Create New Project in Android Studio

Steps

7. Open Android Studio → *New Project*
8. Select → **Empty Activity**
9. Name: **CollegeEventApp**
10. Language: **Java**
11. Minimum SDK: **21 or above**
12. Finish

2. Project Structure

3. XML Layouts

(A) activity_main.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

(B) list_item.xml

```
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/txtEvent"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:padding="12dp"/>
```

(C) activity_second.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/txtSelected"
        android:textSize="20sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
    <Button
        android:id="@+id/btnRegister"
        android:text="Register"
```

```
        android:layout_marginTop="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

4. Java Code

(A) EventAdapter.java – ADAPTER

```
package com.example.collegeeventapp;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.BaseAdapter;
public class EventAdapter extends BaseAdapter {
    Context context;
    String[] events;
    public EventAdapter(Context context, String[] events) {
        this.context = context;
        this.events = events;
    }
    @Override
    public int getCount() {
        return events.length;
    }
    @Override
    public Object getItem(int i) {
        return events[i];
    }
    @Override
    public long getItemId(int i) {
```

```
        return i;
    }
    @Override
    public View getView(int i, View view, ViewGroup parent) {
        if(view == null) {
            view = LayoutInflater.from(context)
                    .inflate(R.layout.list_item, parent, false);
        }
        TextView txt = view.findViewById(R.id.txtEvent);
        txt.setText(events[i]);
        return view;
    }
}
```

(B) MainActivity.java – INTENT + LISTENER

```
package com.example.collegeeventapp;
import android.content.Intent;
import android.os.Bundle;
import android.widget.AdapterView;
import android.widget.ListView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    ListView listView;
    String[] events = {
        "Coding Contest",
        "Technical Quiz",
        "Project Expo",
        "Workshop"
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
        listView = findViewById(R.id.listView);
        EventAdapter adapter =
            new EventAdapter(this, events);
        listView.setAdapter(adapter);
        // EVENT + LISTENER
        listView.setOnItemClickListener(
            new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> parent,
                    android.view.View view,
                    int position, long id) {
                // INTENT
                Intent i = new Intent(
                    MainActivity.this,
                    SecondActivity.class);
                i.putExtra("event",
                        events[position]);
                startActivity(i);
            }
        });
    }
}
```

(C) SecondActivity.java – EVENT HANDLING

```
package com.example.collegeeventapp;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class SecondActivity extends AppCompatActivity {
```

```
    TextView txt;
    Button btn;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
        txt = findViewById(R.id.txtSelected);
        btn = findViewById(R.id.btnRegister);
        // RECEIVE INTENT DATA
        String event =
            getIntent().getStringExtra("event");
        txt.setText("Selected: " + event);
        // LISTENER EVENT
        btn.setOnClickListener(v ->
            Toast.makeText(
                getApplicationContext(),
                "Registered for " + event,
                Toast.LENGTH_LONG
            ).show()
        );
    }
}
```